Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 4

Part A (To be referred by students)

Sequences and Dictionary

SAVE THE FILE AND UPLOAD AS (RollNo_Name_Exp4)

Aim: To understand and utilize Python collections (lists, tuples, sets, and dictionaries) and explore their applications in data science and statistics.

Problem Statement:

Solve the following computational problems by **recognizing** core Python data structures and **apply** them to implement programs

1. Create a list of the first 10 positive integers and print the sum of the even numbers in the list
2. Demonstrate the following operations on a number list using list functions
    a. insert 13 at position 4
    b. Sort the list in ascending order
    c. delete the last element
    d. remove 31
    e. Reverse the list
    f. Append one number to list
    g. extend the list with [20,30,40]
    h. print the number of elements in the list using while
3. Given the tuple (10, 20, 30, 40, 50), use slicing to print a new tuple containing only the last three elements.
4. Create a set of the numbers {1, 2, 3, 4, 5} and compute the intersection with {4, 5, 6, 7}; print the result.
5. Define a dictionary with the keys 'a', 'b', and 'c' and their respective values 1, 2, and 3, then update the value for key 'b' to 10 and print the updated dictionary.
6. Using a list comprehension, create a list of squares for the numbers from 1 to 5 and print the resulting list.
7. Create a tuple with elements 'cat', 'dog', and 'bird', then check and print whether 'dog' is in the tuple.
8. Given the list [1, 1, 2, 3, 5, 8, 13], convert it to a set and print the set's length.
9. Create two dictionaries {'A': [1, 2], 'B': [3, 4]} and {'B': [5, 6], 'C': [7, 8]}, then merge them into one dictionary and print the result.

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

10. For the list [3, 5, 7, 3, 5, 9, 3], create a dictionary to count the occurrences of each element and print the dictionary.
11. Create a set, add member(s) in a set and perform the following operations: intersection of sets, union of sets, set difference, symmetric difference, find length, maximum, minimum value in a set, and clear a set.
12. Create a dictionary with the day no as the key and the day as the value & display it. (ex. 1 – Monday, …)
13. Write a Python program to find the sum of all items in the dictionary.
14. Create a dictionary to keep students' marks, use students' sapid as the key. Perform the task mentioned below on this dictionary.
    a. Display all the keys
    b. Display all the values
    c. Take the sapid as the input and modify the grade given by the user
    d. Take the sapid from the user to remove that user from the dictionary
    e. Give 5 marks as a bonus to all the students and display the new marks
    f. Find the length of the dictionary using the len function
    g. Create a new copy of the dictionary using the copy method
15. Calculate total sales from a product-sales dictionary.
16. Find highest and lowest temperatures in [22, 25, 19, 30, 28, 32, 26].
17. Print the name of the student with the highest score from a list of tuples [(90, 'Alice'), (85, 'Bob'), (88, 'Charlie')].
18. Compute and print the cumulative sum of [2, 4, 6, 8, 10].
19. Create a list where numbers divisible by 3 become "Fizz", divisible by 5 become "Buzz", both become "FizzBuzz", for numbers 1 to 15.
20. Create a list of numbers from 1 to 100 divisible by both 2 and 5.
21. Write a program that takes a dictionary where each value is a list and converts it into a dictionary where each value is a set containing the unique elements from the original list.
22. Write a program that takes two dictionaries and returns a set of keys that are present in the first dictionary but not in the second.
23. Write a function that takes two sets of tuples and returns a tuple containing three sets:
    i. Elements that are in both sets.
    ii. Elements that are only in the first set.
    iii. Elements that are only in the second set.
24. Write a program that takes a list of sets and returns a single set that is the union of all the sets in the list.
25. Write a program that takes a list of tuples and rotates each tuple by a given number of positions. For example, rotating `(1, 2, 3, 4)` by 2 positions would result in `(3, 4, 1,2)`.
26. Write a program that takes a list of tuples where each tuple contains integers and returns the tuple with the highest sum of its elements.

Topic covered: List, Tuple, Set & Dictionary

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

Learning Objective: Learner would be able to

1. Recognize core python data structures & apply them to solve problems
2. Solve the problems using sequences like list, tuple, set
3. Understand dictionary and solving problems using dictionary
4. Perform various operations (using functions) on sequences and dictionary

Theory:

Lists:

Lists are used to store multiple items in a single variable. Lists are created using square brackets. List items are ordered, changeable, and allow duplicate values. List can contain strings, integers, as well as objects. It can be used to implement stacks and queues.
List items are indexed, the first item has index [0], the second item has index [1] etc.

- **Create List:**

A tuple a list is created by placing elements inside square brackets [], separated by commas. A list can have any number of items and they may be of different types (integer, float, string, etc.). A list can also have another list as an item. This is called a nested list.

```
# list of integers
my_list = [1, 2, 3]
print(my_list)

# empty list
my_list = []
print(my_list)
# list with mixed data types
my_list = [1, "Hello", 3.4]
print(my_list)

# nested list
```

```
my_list = ["mouse", [8, 4, 6], ['a']]
print(my_list)
```

Output:

[1, 2, 3]
[]
[1, 'Hello', 3.4]
['mouse', [8, 4, 6], ['a']]

- **Access Items:**

  List Index:

  We can use the index operator [] to access an item in a list. In Python, indices start at 0. So, a list having 5 elements will have an index from 0 to. Python allows negative indexing for its sequences. The index of -1 refers to the last item, -2 to the second last item and so on. We can access a range of items in a list by using the slicing operator : .

```
my_list = ['p', 'r', 'o', 'b', 'e']

# first item
print(my_list[0])  # p

# third item
print(my_list[2])  # o

# fifth item
print(my_list[4])  # e

# Nested List
n_list = ["Happy", [2, 0, 1, 5]]
```

```
# Nested indexing
print(n_list[0][1])

print(n_list[1][3])

# Error! Only integer can be used for indexing
print(my_list[4.0])

# Negative indexing in lists
my_list = ['p','r','o','b','e']

# last item
print(my_list[-1])

# fifth last item
print(my_list[-5])

# List slicing in Python

my_list = ['p','r','o','g','r','a','m','i','z']

# elements from index 2 to index 4
print(my_list[2:5])
# elements from index 5 to end
print(my_list[5:])

# elements beginning to end
print(my_list[:])
```

```
Output:
p
o
e
a
5
Traceback (most recent call last):
File "<string>", line 21, in <module>
TypeError: list indices must be integers or slices, not float
e
p
['o', 'g', 'r']
['a', 'm', 'i', 'z']
['p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z']
```

- **Update Items:**

  We can use the assignment operator = to change an item or a range of items.

  We can add one item to a list using the append() method or add several items using the extend() method.

  We can also use + operator to combine two lists. This is also called concatenation.

  The * operator repeats a list for the given number of times.

  We can insert one item at a desired location by using the method insert() or insert multiple items by squeezing it into an empty slice of a list.

```
# Correcting mistake values in a list
odd = [2, 4, 6, 8]


# change the 1st item
odd[0] = 1
```

```
print(odd)

# change 2nd to 4th items
odd[1:4] = [3, 5, 7]
print(odd)

# Appending and Extending lists in Python
odd = [1, 3, 5]
odd.append(7)
print(odd)

odd.extend([9, 11, 13])
print(odd)

# Concatenating and repeating lists
odd = [1, 3, 5]
print(odd + [9, 7, 5])

print(["re"] * 3)
# Demonstration of list insert() method
odd = [1, 9]
odd.insert(1,3)
print(odd)

odd[2:2] = [5, 7]
print(odd)
```

Output:

[1, 4, 6, 8]

```
[1, 3, 5, 7]
[1, 3, 5, 7]
[1, 3, 5, 7, 9, 11, 13]
[1, 3, 5, 9, 7, 5]
['re', 're', 're']
[1, 3, 9]
[1, 3, 5, 7, 9]
```

- **Remove Item:**

  We can delete one or more items from a list using the Python del statement. It can even delete the list entirely.

```
# Deleting list items
my_list = ['p', 'r', 'o', 'b', 'l', 'e', 'm']

# delete one item
del my_list[2]

print(my_list)

# delete multiple items
del my_list[1:5]

print(my_list)

# delete the entire list
del my_list

# Error: List not defined
print(my_list)
```

Output:

['p', 'r', 'b', 'l', 'e', 'm']

['p', 'm']

Traceback (most recent call last):

File "<string>", line 21, in <module>

NameError: name 'my_list' is not defined

We can use remove() to remove the given item or pop() to remove an item at the given index.

The pop() method removes and returns the last item if the index is not provided. This helps us implement lists as stacks (first in, last out data structure).

And, if we have to empty the whole list, we can use the clear() method.

```
my_list = ['p','r','o','b','l','e','m']
my_list.remove('p')

# Output: ['r', 'o', 'b', 'l', 'e', 'm']
print(my_list)

# Output: 'o'
print(my_list.pop(1))

# Output: ['r', 'b', 'l', 'e', 'm']
print(my_list)

# Output: 'm'
print(my_list.pop())

# Output: ['r', 'b', 'l', 'e']
```

```
print(my_list)


my_list.clear()

# Output: []
print(my_list)
```

Output:

['r', 'o', 'b', 'l', 'e', 'm']
o
['r', 'b', 'l', 'e', 'm']
m
['r', 'b', 'l', 'e']
[]

We can also delete items in a list by assigning an empty list to a slice of elements. The sliced elements are removed from the original list.

```
my_list = ['p','r','o','b','l','e','m']
print(my_list[2:3])
my_list[2:3] = []
print(my_list)

print(my_list[2:5])
my_list[2:5] = []
print(my_list)
```

Output:

['o']
['p', 'r', 'b', 'l', 'e', 'm']

```
['b', 'l', 'e']
['p', 'r', 'm']
```

- **List Operations:**

1. List Membership Test:

We can test if an item exists in a list or not, using the keyword in.

```
my_list = ['p', 'r', 'o', 'b', 'l', 'e', 'm']

# Output: True
print('p' in my_list)

# Output: False
print('a' in my_list)

# Output: True
print('c' not in my_list)
```

Output:

True

False

True

2. Iterating through a List:

We can use a for loop to iterate through each item in a list.

```
for fruit in ['apple','banana','mango']:
    print("I like",fruit)
```

Output:

I like apple

```
I like banana
I like mango
```

- **List Comprehension:**

  List comprehension is an elegant and concise way to create a new list from an existing list in Python.

  A list comprehension consists of an expression followed by **for statement** inside square brackets.

```
pow2 = [2 ** x for x in range(10)]
print(pow2)

Output:
[1, 2, 4, 8, 16, 32, 64, 128, 256, 512]
```

  A list comprehension can optionally contain more for or **if statements**.

```
pow2 = [2 ** x for x in range(10) if x > 5]
print(pow2)
odd = [x for x in range(20) if x % 2 == 1]
print(odd)
z=[x+y for x in ['Python ','C '] for y in ['Language','Programming']]
print(z)

Output:
[64, 128, 256, 512]
[1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
['Python Language', 'Python Programming', 'C Language', 'C Programming']
```

Tuple:

A tuple is a sequence of **immutable** (cannot be changed) Python objects. Tuples use parentheses. Tuple are used to store sequence of python objects which are static in nature. Just like a list, a tuple contains a sequence of objects **in order** but once created a tuple, it cannot be changed anything about it. Tuple indices start from 0. Since tuples are indexed, they can have items with the same value.

- **Create a Tuple:**

  A tuple can have any number of items and they may be of different types (integer, float, list, string, etc.).

- **Create Tuple with One Item:**

  To create a tuple with only one item, you have to add a comma after the item, otherwise Python will not recognize it as a tuple.

```
tup1 = ('physics', 'chemistry', 1997, 2000);
tup2 = (1, 2, 3, 4, 5);
tup3 = "a", "b", "c", "d";
print(tup1)
print(tup2)
print(tup3)
tup4 = ()
print(tup4)
tup4 = (50, );
print(tup4)
```

Output: Tuples are ordered and items cannot be changed once added.

('physics', 'chemistry', 1997, 2000)

(1, 2, 3, 4, 5)

('a', 'b', 'c', 'd')

()

(50,)

- **Access Items:**

  To access values in tuple, use the square brackets for slicing along with the index or indices to obtain value available at that index.

```
tup1 = ('physics', 'chemistry', 1997, 2000)
tup2 = (1, 2, 3, 4, 5, 6, 7)
print ("tup1[0]: ", tup1[0])
print ("tup2[1:5]: ", tup2[1:5])
print(tup2[-1])
print(tup2[-6])
```

Output:

tup1[0]: physics

tup2[1:5]: (2, 3, 4, 5)

7

2

- **Update Items:**

  Tuples are immutable which means you cannot update or change the values of tuple elements. You are able to take portions of existing tuples to create new tuples.

```
tup1 = (12, 34.56)
tup2 = ('abc', 'xyz')
# Following action is not valid for tuples
# tup1[0] = 100
# So let's create a new tuple as follows
tup3 = tup1 + tup2
print (tup3)
```

Output:

(12, 34.56, 'abc', 'xyz')

- **Remove Item:**

  Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded. To explicitly remove an entire tuple, just use the del statement.

```
tup = ('physics', 'chemistry', 1997, 2000)
print (tup)
del (tup)
print (tup)
```

Output:

('physics', 'chemistry', 1997, 2000)

Traceback (most recent call last):

File "<string>", line 4, in <module>

NameError: name 'tup' is not defined

- **Tuple Operations:**

  3. Tuple Membership Test:

  We can test if an item exists in a tuple or not, using the keyword in.

```
# Membership test in tuple
my_tuple = ('a', 'p', 'p', 'l', 'e',)

# In operation
print('a' in my_tuple)
print('b' in my_tuple)

# Not in operation
print('g' not in my_tuple)
```

Output:

True

False

True

4. Iterating through a Tuple:

We can use a for loop to iterate through each item in a tuple.

```
# Using a for loop to iterate through a tuple
for name in ('John', 'Kate'):
    print("Hello", name)
```

Output:

Hello John

Hello Kate

5. Repetition:

Repeat the elements in a tuple for a given number of times using the * operator.

6. Concatenation:

Use + operator to combine two tuples. This is called concatenation.

7. Length:

Count the number of elements in the tuple.

```
# Concatenation
t = (1, 2, 3)
p =(4, 5, 6)
print(t + p)
```

```
# length
print(len(t))


# Repeat
# Output: ('Repeat', 'Repeat', 'Repeat')
print(("Repeat",) * 3)
```

Output:

(1, 2, 3, 4, 5, 6)

3

('Repeat', 'Repeat', 'Repeat')

Set:

A set is a collection which is **unordered** and **unindexed**. In Python sets are written with **curly brackets**. Every element is **unique (no duplicates)** and must be immutable (which cannot be changed). It can have any number of items and they may be of **different types (integer, float, tuple, string etc.)**. The sets in Python are typically used for mathematical operations like union, intersection, difference and complement etc.

- **Create a Set:**

```
thisset = {"apple", "banana", "cherry"}
print(thisset)


Output: Sets are unordered, so the items will appear in a random order.
{'banana', 'cherry', 'apple'}
```

- **Access Items:**

You cannot access items in a set by referring to an index, since sets are unordered the items has no index. But you can loop through the set items using a for loop, or ask if a specified value is present in a set, by using thein keyword.

```
thisset = {"apple", "banana", "cherry"}
print(thisset)

for x in thisset: print(x)

print("banana" in thisset)
```

Output:

{'cherry', 'banana', 'apple'}
cherry
banana
apple
True

- **Update Items:**

  Once a set is created, you cannot change its items, but you can add new items.

- **Add Items:**

  To add one item to a set use the add() method.

  To add more than one item to a set use the update() method.

```
thisset = {"apple", "banana", "cherry"}
thisset.add("orange")
print(thisset)
thisset.add("apple")
print(thisset)
```

Output:

{'apple', 'cherry', 'orange', 'banana'}

{'apple', 'cherry', 'orange', 'banana'}

Add multiple items to a set, using the update() method:

```
thisset = {"apple", "banana", "cherry"}
thisset.update(["orange", "mango", "grapes"])
print(thisset)
```

Output:

{'apple', 'cherry', 'orange', 'banana'}

- **Length of a Set:**

  To determine how many items a set has, use the len() method.

```
thisset = {"apple", "banana", "cherry"}
print(len(thisset))
```

Output:

3

- **Remove Item:**

  To remove an item in a set, use the remove(), or the discard() method.

```
# remove()
thisset = {"apple", "banana", "cherry"}
thisset.remove("banana")
print(thisset)

thisset.remove("banana")
print(thisset)
```

Output:

{'cherry', 'apple'}

```
Traceback (most recent call last):
  File "<string>", line 6, in <module>
KeyError: 'banana'
```

Note: If the item to remove does not exist, remove() will raise an error.

Note: If the item to remove does not exist, discard() will NOT raise an error.

```
# discard()
thisset = {"apple", "banana", "cherry"}
thisset.remove("banana")
print(thisset)
thisset.remove("banana")
print(thisset)

Output:
{'apple', 'cherry'}
{'apple', 'cherry'}
```

Note: Sets are unordered, so when using the gets removed. pop() method, you will not know which item that gets removed.

You can also use the pop(), method to remove an item, but this method will remove last item. Since set is unordered, so you will not know what item that gets. The return value of the pop() method is the removed item.

```
thisset = {"apple", "banana", "cherry"}
x = thisset.pop()
print(x)
print(thisset)

Output:
banana
```

```
{'cherry', 'apple'}
```

The clear() method empties the set:

```
thisset = {"apple", "banana", "cherry"}
thisset.clear()
print(thisset)
thisset.add("organge")
print(thisset)

Output:
set()
{'organge'}
```

The del() method will delete the set comple:

```
thisset = {"apple", "banana", "cherry"}
del thisset
print(thisset)

Output:
Traceback (most recent call last):
  File "<string>", line 5, in <module>
NameError: name 'thisset' is not defined
```

Dictionary:

A dictionary is a collection which is unordered, changeable and indexed. In Python dictionaries are written with curly brackets, and they have keys and values. It is a collection which is ordered (Python version 3.7 and above) and changeable (meaning that we can change, add or remove items after the dictionary has been created). No duplicate members.

- **Create a Dictionary:**

```
thisdict = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}
print(thisdict)
```

Output: Dictionary are ordered.

{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}

- **Access Items:**

  You can access the items of a dictionary by referring to its key name, inside square brackets.

```
# Get the value of the "model" key:
thisdict = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}
x = thisdict["model"]
print(x)
```

Output:

Mustang

get() : There is also a method called get() that will give you the same result:

```
# Get the value of the "model" key:
thisdict = {
"brand": "Ford",
```

```
"model": "Mustang",
"year": 1964
}
x = thisdict.get("model")
print(x)
```

Output:

Mustang

- **Update and Add Items:**

  Dictionaries are mutable. We can add new items or change the value of existing items using an assignment operator.

  If the key is already present, then the existing value gets updated. In case the key is not present, a new (key: value) pair is added to the dictionary.

```
# Changing and Adding Dictionary elements:
thisdict = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}
print(thisdict)
# update value
thisdict['year'] = 1966
print(thisdict)
# add item
thisdict["color"] = 'White'
print(thisdict)
```

Output:

{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}

{'brand': 'Ford', 'model': 'Mustang', 'year': 1966}

{'brand': 'Ford', 'model': 'Mustang', 'year': 1966, 'color': 'White'}

- **Built-in functions of Dictionary:**

  all()

  Return True if all keys of the dictionary are True (or if the dictionary is empty).

  any()

  Return True if any key of the dictionary is true. If the dictionary is empty, return False.

  len()

  Return the length (the number of items) in the dictionary.

  cmp()

  Compares items of two dictionaries. (Not available in Python 3)

  sorted()

  Return a new sorted list of keys in the dictionary.

```
# Dictionary Built-in Functions
squares = {0: 0, 1: 1, 3: 9, 5: 25, 7: 49, 9: 81}

# Output: False
print(all(squares))

# Output: True
print(any(squares))

# Output: 6
print(len(squares))

# Output: [0, 1, 3, 5, 7, 9]
```

```
print(sorted(squares))
```

Output:

False

True

6

[0, 1, 3, 5, 7, 9]

- **Remove Item:**

  We can remove a particular item in a dictionary by using the **pop()** method. This method removes an item with the provided key and returns the value.

  The **popitem()** method can be used to remove and return an arbitrary (key, value) item pair from the dictionary.

  All the items can be removed at once, using the **clear()** method.

  We can also use the **del** keyword to remove individual items or the entire dictionary itself.

```
# Removing elements from a dictionary

# create a dictionary
squares = {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# remove a particular item, returns its value
# Output: 16
print(squares.pop(4))

# Output: {1: 1, 2: 4, 3: 9, 5: 25}
print(squares)

# remove an arbitrary item, return (key,value)
# Output: (5, 25)
```

```
print(squares.popitem())

# Output: {1: 1, 2: 4, 3: 9}
print(squares)

# remove all items
squares.clear()
# Output: {}
print(squares)

# delete the dictionary itself
del squares

# Throws Error
print(squares)
```

Output:

```
16
{1: 1, 2: 4, 3: 9, 5: 25}
(5, 25)
{1: 1, 2: 4, 3: 9}
{}
Traceback (most recent call last):
  File "<string>", line 30, in <module>
NameError: name 'squares' is not defined
```

- **Looping through Dictionary:**

  A dictionary can be iterated using the for loop. If you want to get both keys and the values in the output. You just have to add the keys and values as the argument of the print

statement in comma separation. After each iteration of the for loop, you will get both the keys its relevant values in the output.

```
# The output shows the given key is related to the given value in the output.
dict={'Name':'Zara','Age':7,'Class':'First'}
for key, value in dict.items():
    print(key, '-',value)
```

Output:

Name - Zara

Age - 7

Class - First

- **Python Dictionary Comprehension:**

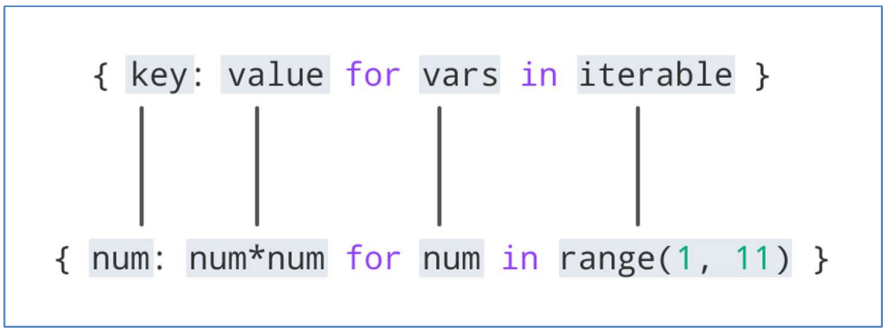
  Python Dictionary Comprehension helps to create a new dictionary from an iterable in Python.

  Dictionary comprehension consists of an expression pair (**key: value**) followed by a for statement inside curly braces {}.

  The minimal syntax for dictionary comprehension is:

```
dictionary = {key: value for vars in iterable}
```

Let's compare this syntax with dictionary comprehension from the above example.



```
# Dictionary Comprehension to create square of numbers from 0 to 5
squares = {x: x*x for x in range(6)}
print(squares)
```

Output:

{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

Conditionals in Dictionary Comprehension:

We can further customize dictionary comprehension by adding conditions to it.

The syntax for dictionary comprehension with conditionals is:

```
dictionary = {key: true_value for vars in iterable   if  condition }
```

```
dictionary = {key: ( true_value if condition else false_value ) for vars in iterable}
```

```
# Apply dictionary comprehension to create squares of only odd numbers from 1 to 11

odd_squares = {x: x*x for x in range(11) if x % 2 == 1}
print(odd_squares)
```

Output: {1: 1, 3: 9, 5: 25, 7: 49, 9: 81}

```
# apply dictionary comprehension to pick up only items having even age values
original_dict = {'jack': 38, 'michael': 48, 'guido': 57, 'john': 33}

even_dict = {k: v for (k, v) in original_dict.items() if v % 2 == 0}
print(even_dict)
```

Output: {'jack': 38, 'michael': 48}

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 4

Part B (to be completed by students)

Sequences and Dictionary

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp4)
3. In the first cell, the student must write his/her Name, roll no and class in the form of comments
4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp4.pdf).
   Note: - To download as a PDF, go to File and choose Print Preview. This will open a new tab. Now, press Ctrl + P and opt to save as PDF.
6. Upload the pdf on the web portal